

컴퓨터비전(AI응용)

Transformer_ViT



Self-Attention 메커니즘 완벽 이해

Query, Key, Value의 역할과 Scaled Dot-Product Attention 수식을 직관적으로 이해합니다.



Multi-Head Attention & Transformer 구조

다중 관점 학습의 원리와 Encoder-Decoder 구조, Positional Encoding을 파악합니다.



NLP 응용: BERT

사전학습과 파인튜닝의 개념을 익히고, 감정 분석(SST-2) 실습을 진행합니다.



Vision 응용: ViT (Vision Transformer)

이미지를 패치로 처리하는 혁신적 방법을 배우고, 콩 질병 분류(97% 정확도)를 실습합니다.



성능 비교 분석

LSTM vs Transformer, CNN vs ViT의 장단점과 성능 차이를 명확히 비교합니다.



NLP & Vision 통합

텍스트와 이미지 처리의
경계를 허무는 통합적 시각

왜 Transformer가 필요했나?

순차적 처리의 한계와 병렬 처리 혁신

! RNN/LSTM의 3가지 한계

1. 순차 처리 → 병렬화 불가

이전 단계 계산이 완료되어야 다음 단계 시작 가능.
GPU의 병렬 처리 능력을 활용하지 못해 학습 속도가 느림.

2. 장거리 의존성 (Long-term Dependency)

문장이 길어질수록 앞부분 정보 소실.
Gradient Vanishing 문제로 인해 문맥 유지가 어려움.

3. 정보 병목 현상

모든 정보를 고정된 크기의 Hidden State 벡터 하나에 압축해야 하므로
정보 손실 발생.

⚡ Transformer의 혁신

1. 완벽한 병렬화 (Parallelization)

모든 위치의 단어를 동시에 입력받아 계산.
GPU 활용을 극대화하여 학습 속도 10배 이상 향상.

2. 직접 연결 (Direct Connection)

Self-Attention을 통해 모든 위치가 서로 직접 참조.
거리에 상관없이 정보 전달 가능

3. 무한한 메모리 접근

정보를 압축하지 않고,
Attention 메커니즘으로 필요한 정보만 선택적으로 가져와 사용.

구조적 차이 비교



RNN: 순차적 처리 (병목 발생)

VS



Transformer: 완전 연결 (병렬 처리)

Self-Attention의 핵심: Query, Key, Value

데이터베이스 검색 비유를 통한 직관적 이해 (Query, Key, Value)

데이터베이스 검색 비유

1. Query (Q) - 질문

"내가 지금 찾고자 하는 정보는 무엇인가?"
현재 위치(토큰)에서 다른 위치의 정보를 찾기 위해 던지는 질문 벡터.

비유: 검색창에 입력하는 검색어 "스마트폰"

2. Key (K) - 색인

"나는 어떤 정보를 가지고 있는가?"
각 위치의 정보가 자신을 식별할 수 있도록 제공하는 색인 벡터.

비유: 각 웹페이지의 제목/태그 "아이폰 15 리뷰"

3. Value (V) - 실제 내용

"전달할 실제 내용은 무엇인가?"
Query와 Key가 일치할 때 최종적으로 가져올 실제 정보 벡터.

비유: 클릭해서 들어간 웹페이지의 본문 내용

실제 문장 예시

"The animal didn't cross the street because it was too tired" **it**

'it'이 무엇을 가리키는지 찾는 과정

? Query (it): "나는 누구를 가리키는 대명사인가?"

Q Key (모든 단어): 각 단어가 자신의 특성(명사, 동사 등)을 노출

✓ 매칭 결과:
'it'의 Query와 'animal'의 Key가 가장 높은 유사도(Attention Score)를 가짐

↓ Value 가져오기: 'animal'의 Value 정보(동물, 주어 등)를 'it'에 반영

→ 결과적으로 기계는 "it = animal"임을 학습하게 됨

Self-Attention의 핵심: Query, Key, Value

Self-Attention 계산 5단계

입력 벡터에서 최종 Attention 출력까지의 수학적 처리 과정

1

Q, K, V 생성 (Linear Projection)

입력 벡터 X에 학습 가능한 가중치 행렬을 곱해 3개의 벡터 생성

$$X \times W_Q, W_K, W_V$$

2

Attention Score 계산

Query와 Key의 내적을 통해 두 단어 간의 연관성(유사도) 계산

$$\text{Score} = Q \cdot K^T$$

3

Scaling (스케일링)

차원 크기(d_k)의 제곱근으로 나누어 값의 크기를 조정 (Gradient 안정화)

$$\text{Score} / \sqrt{d_k}$$

4

Softmax 적용

점수를 0~1 사이의 확률값으로 변환 (전체 합 = 1)

$$\text{softmax}(\text{Scaled Score})$$

5

Value 가중합 (Weighted Sum)

Attention 확률을 가중치로 하여 Value 벡터들의 합을 계산

$$\sum (\text{Weights} \times V)$$

간단한 숫자 예시 ($d_k = 4, \sqrt{d_k} = 2$)

Q (Query)

[1, 0, 2, 1]

K (Key)

[1, 1, 0, 1]

↓ 내적 (Dot Product)

Score

$$1 \times 1 + 0 \times 1 + 2 \times 0 + 1 \times 1 = 2$$

÷ 2

Scaled

1.0

↓ Softmax (가정: 다른 단어 점수와 비교)

Attention Weights

나(Me)

0.7

너(You)

0.2

그(Him)

0.1

$$\text{최종 벡터} = 0.7 \times V_1 + 0.2 \times V_2 + 0.1 \times V_3$$

가장 관련성 높은 V_1 의 정보가 70% 반영됨.
→ "나"에 대한 정보가 집중적으로 전달됨.

$$\text{Attention}(Q, K, V) = \text{softmax} \frac{QK^T}{\sqrt{d_k}} V$$

기호 (Symbol)	의미 (Meaning)	차원 (Dimension)	역할 (Role)
Q	Query (질문)	$[n \times d_k]$	현재 위치에서 찾고자 하는 정보의 기준 벡터
K	Key (색인)	$[n \times d_k]$	각 위치가 가진 정보를 식별하기 위한 색인 벡터
V	Value (값)	$[n \times d_v]$	실제 전달할 내용을 담고 있는 정보 벡터
QK^T	Attention Score	$[n \times n]$	모든 토큰 쌍 간의 유사도 (내적 값)
softmax	확률 변환	$[n \times n]$	점수를 확률 분포(합=1)로 정규화

Scaled Dot product Attention 수식

이 메커니즘은 모든 위치 간 관계를 동시에 계산하므로 $O(n^2)$ 복잡도를 가지지만, 완벽한 병렬 처리가 가능하여 GPU에서 매우 빠릅니다.

Step 1: Attention Score 계산

Query와 Key의 내적으로 유사도 계산

$$\text{Score} = QK^T$$

행렬 차원: $(\text{seq_len}, d_k) \times (d_k, \text{seq_len})$
 $= (\text{seq_len}, \text{seq_len})$

Step 2: Scaling

차원의 제곱근으로 나누어 gradient 안정화

$$\text{Scaled Score} = \frac{QK^T}{\sqrt{d_k}}$$

d_k 가 크면 내적 값이 커져 softmax가 극단적 확률을 출력하므로 스케일링 필수

Step 3: Softmax & Weighted Sum

확률 분포로 변환 후 Value와 가중합

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

최종 출력 차원: $(\text{seq_len}, d_v)$

왜 Scaling($\sqrt{d_k}$)이 필요한가?

문제점: 차원(d_k)이 커질수록 내적 값(QK^T)이 매우 커짐.

결과:

Softmax 함수가 극단적인 값(0 또는 1)을 출력하게 되어, 역전파 시 **Gradient Vanishing**(기울기 소실) 발생.

시간 복잡도 (Time Complexity)

복잡도: $O(n^2 \cdot d)$

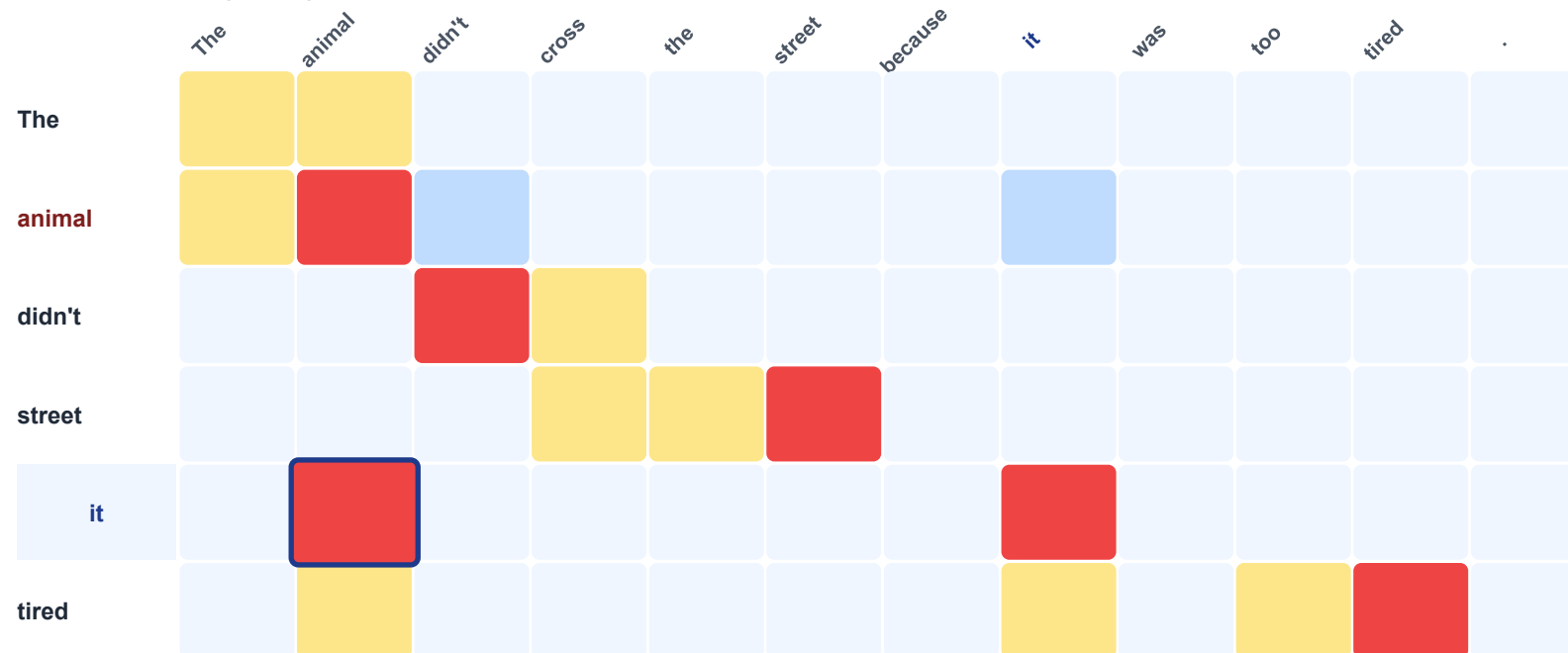
n : 시퀀스 길이 (Sequence Length)

Attention 시각화 예시

실제 문장에서 단어 간의 관계(Self-Attention)를 어떻게 학습하는가?

"The **animal** didn't cross the street because **it** was too tired"

Attention Map (Query → Key)



Attention Weight 범례

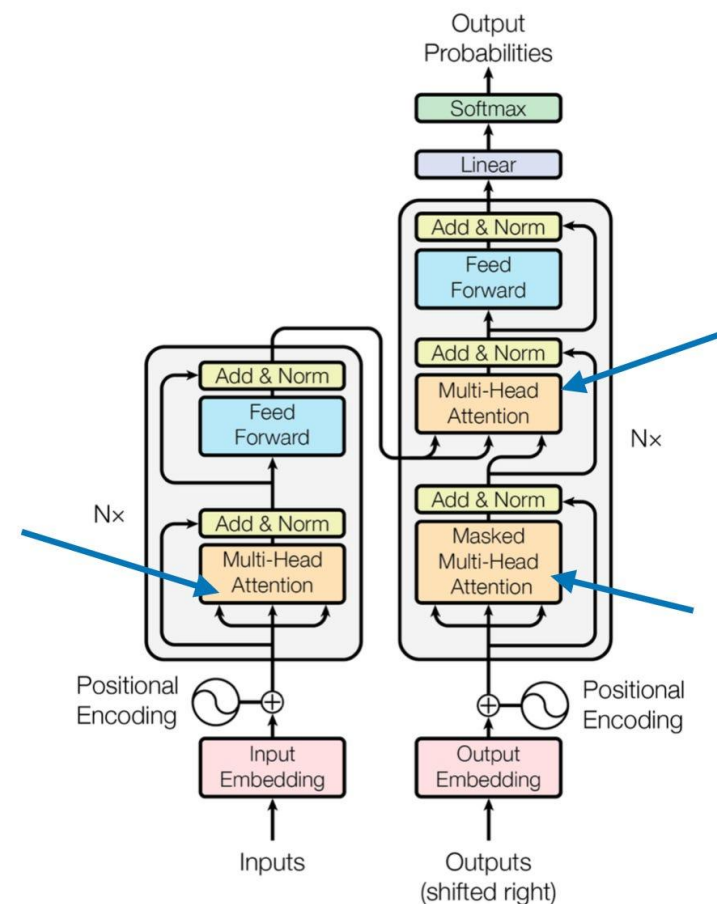


모델의 해석 능력

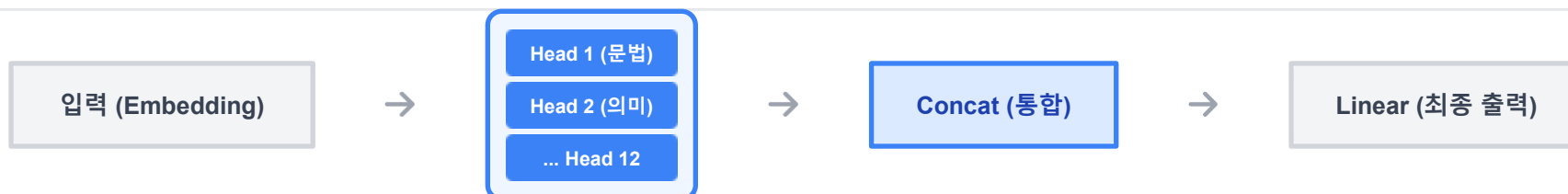
- ✓ **대명사 해결 (Coreference Resolution):**
모델은 'it'이 가리키는 대상이 'street'가 아니라 'animal'임을 문맥('tired')을 통해 파악함.
- ✓ **높은 가중치 (0.9):**
'it'의 Query가 'animal'의 Key와 매우 유사하여, 'animal'의 정보를 90% 이상 가져옴.
- ✓ **결과:**
기계가 문장의 의미적 구조를 스스로 학습함을 증명

Multi-Head Attention은 입력 Query, Key, Value를 각각 여러 개의 "head"로 분할하고, 각 head마다 독립적으로 attention을 계산한 후 결과를 결합하는 방식으로 작동합니다.

이는 모델이 여러 표현 공간에서 동시에 정보를 집중할 수 있도록 합니다.



처리 흐름도



왜 Multi-Head가 필요할까?

단일 관점의 한계와 다중 관점 학습의 필요성

⚠ 단일 Attention의 한계

다양한 관계 동시 포착 불가

하나의 문장 안에는 문법적, 의미적, 위치적 등 여러 종류의 관계가 혼재되어 있음.
단일 Head는 이 모든 것을 한 번에 학습하기 어렵습니다.

"The teacher taught the student in the classroom"

teacher ← taught (주어)

taught → student (목적어)

student → classroom (장소)

→ 하나의 Attention으로 이 모든 화살표를 그리기 어려움!

≡ Multi-Head의 해결책

여러 관점 동시 학습 (Multi-View)

입력을 여러 개의 하위 공간(Subspace)으로 나누어,
각 Head가 서로 다른 관점의 특징을 독립적으로 학습합니다.

📷 Head 1: 문법 구조

📷 Head 2: 의미 유사도

📷 Head 3: 위치 정보

📷 Head 4: 화자 의도

📷 12대 카메라 비유

단일 Head는 1대의 카메라로 촬영하는 것과 같지만,
Multi-Head는 12대의 카메라가 다양한 각도에서 피사체를 동시에 촬영하여
더 입체적이고 풍부한 정보를 얻는 것과 같습니다.

각 attention head는 서로 다른 가중치 행렬을 학습하므로, 문장 내에서 다양한 유형의 관계에 집중할 수 있습니다. 이는 단일 attention으로는 포착하기 어려운 복잡한 문맥 이해를 가능하게 합니다.



구문적 관계

한 head는 주어-동사, 수식어-피수식어 등 문법적 구조를 파악하는 데 집중할 수 있습니다.



의미론적 유사성

다른 head는 동의어나 유사한 의미를 가진 단어들을 연결하여 의미적 문맥을 형성할 수 있습니다.



대명사 및 개체명 참조

"그녀", "그것"과 같은 대명사가 가리키는 실제 개체나 명명된 개체(예: 회사, 사람 이름) 간의 관계를 추적할 수 있습니다.



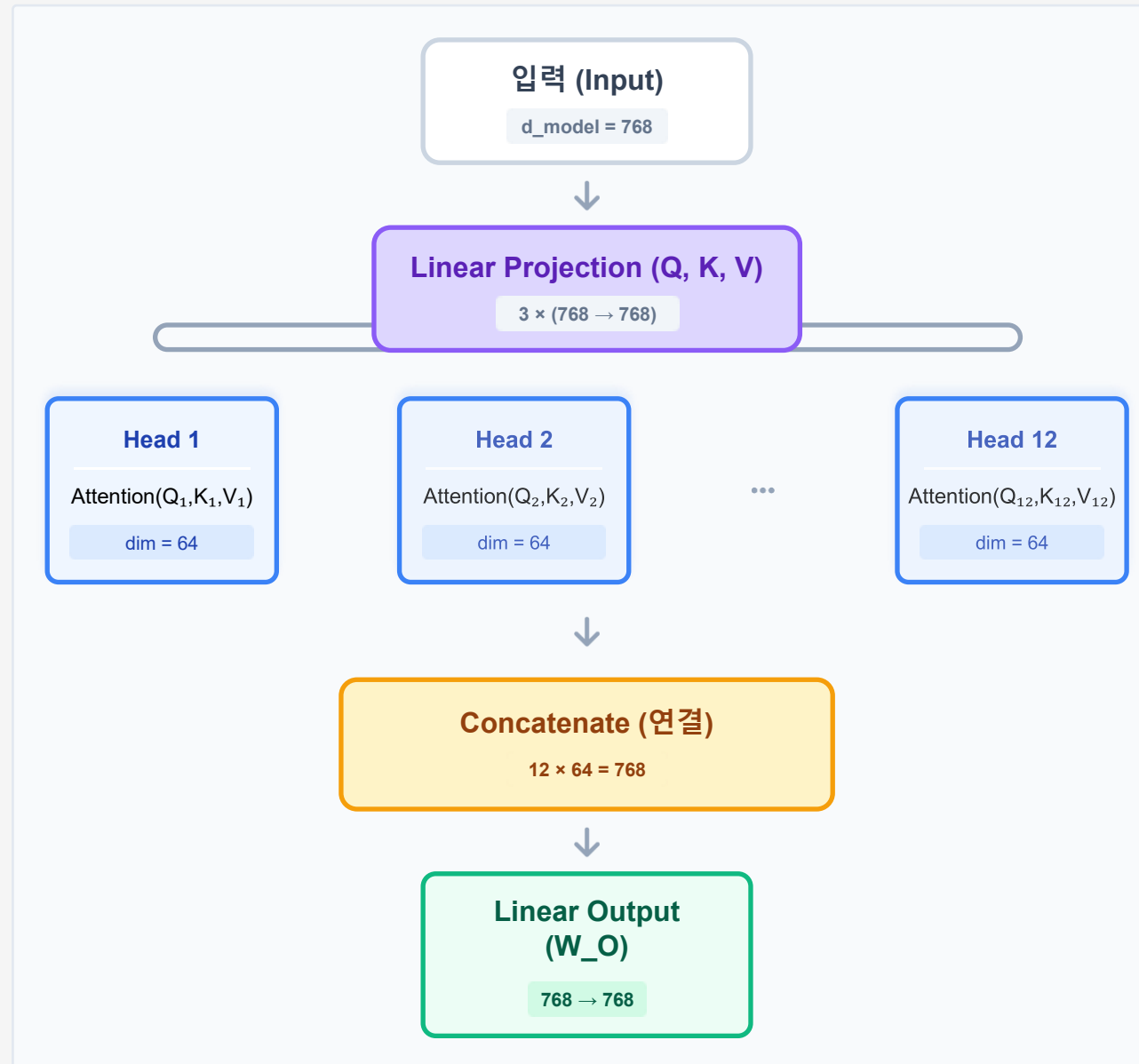
장거리 의존성

문장의 앞부분과 뒷부분에 있는 단어들 사이의 숨겨진 관계나 맥락을 이해하는 데 특화될 수 있습니다.



Multi-Head Attention 구조

입력 분할부터 병렬 처리, 최종 통합까지의 전체 흐름도



1 선형 변환 및 분할

입력 Q, K, V 를 각각 h 개의 다른 선형 변환(W_Q, W_K, W_V 행렬)을 통해 저차원 공간으로 투영하고, 각 head에 맞게 분할합니다.

1

$$Q_i = QW_{Q_i}, \quad K_i = KW_{K_i}, \quad V_i = VW_{V_i}$$

여기서 $W_{Q_i}, W_{K_i}, W_{V_i}$ 는 i 번째 head의 가중치 행렬입니다.

2 각 Head별 Scaled Dot-Product Attention 계산

분할된 Q_i, K_i, V_i 를 사용하여 각 head에서 독립적으로 Scaled Dot-Product Attention을 수행합니다.

2

$$\text{Attention}_i(Q, K, V) = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i$$

3 결과 Concatenation 및 최종 투영

각 head의 attention 결과를 모두 연결(concatenate)한 다음, 하나의 최종 선형 변환(W_O 행렬)을 적용하여 원래 차원으로 복원합니다.

3

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

여기서 $\text{head}_i = \text{Attention}_i(Q, K, V)$ 입니다.

 BERT-base 예시

전체 차원 (d_model)

768

Head 개수 (h)

12

Head별 차원 (d_k)

64

총 파라미터

~110M

 핵심 포인트

- ✓ 독립적 학습:
각 Head는 서로 다른 가중치 행렬(W_Q , W_K , W_V)을 가집니다.
- ✓ 병렬 처리:
12개의 Attention 계산은 동시에 수행되므로 GPU 효율이 높음
- ✓ 차원 유지:
분할 후 다시 Concat하므로 입력과 출력의 차원이 동일합니다.

- Google의 BERT와 같은 트랜스포머 모델은 Multi-Head Attention을 적극 활용
- BERT-base 모델은 일반적으로 12개의 attention head를 사용하며, 각 head의 차원은 64 ($d_{\text{model}}/h = 768/12$)임
- 이는 모델이 매우 다양한 언어적 패턴을 동시에 학습하고 통합할 수 있도록 하여 뛰어난 성능을 달성하는 핵심 요인 중 하나임

단일 / 다중 head 성능 비교

실험 결과 (WMT 2014)

Base Model 기준

Head 수 (h)	BLEU Score	학습 시간 (h)	비고
1	24.5	2.5	기준 (Baseline)
4	26.1	2.8	성능 향상 시작
8	27.3	3.0	유의미한 향상
12	27.8	3.2	최적 성능 (BERT-Base)
16	27.9	3.5	수익 체감 (Diminishing Return)

Head 수에 따른 성능 변화 (BLEU Score)



각 Head가 학습하는 패턴의 다양성



Head 1-4: 문법 구조

주어-동사, 동사-목적어 등 문장 성분 간의 의존 관계를 주로 포착합니다.

Syntactic



Head 5-8: 의미적 유사도

동의어, 반의어, 또는 문맥상 의미가 통하는 단어끼리 연결합니다.

Semantic



Head 9-12: 위치 및 기타

인접한 단어나 문장의 시작/끝과 같은 위치 정보를 학습합니다.

Positional

위치 정보 인코딩 (Positional Encoding)

벡터 합산 메커니즘



→ Self-Attention의 맹점

Attention 메커니즘은 단어의 순서 정보를 고려하지 않습니다. "개가 고양이를 쫓았다"와 "고양이가 개를 쫓았다"가 같은 attention score를 가질 수 있습니다.

→ 사인/코사인 함수 활용

각 위치 pos 와 차원 i 에 대해 고정된 사인·코사인 값을 더해 위치 정보를 주입합니다.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

→ 학습 불필요한 고정 패턴

사인·코사인 함수는 학습 없이 상대적 위치 관계를 표현할 수 있어 임의의 길이 시퀀스에 일반화됩니다. 최근 모델(BERT, GPT)은 학습 가능한 positional embedding도 사용합니다.

위치 정보 인코딩 (Positional Encoding)

Self-Attention의 치명적 약점: 순서를 모른다!

⚠ Self-Attention의 순서 무관성

1. 집합 연산 (Permutation Invariant)

Self-Attention은 입력 단어들의 순서가 바뀌어도 결과가 동일하게 계산됨.
즉, {A, B, C}와 {C, B, A}를 구별하지 못함.

2. 의미 왜곡 문제

"I love you"

=

"you love I"

순서 정보가 없으면 주어와 목적어가 뒤바뀐 문장도 동일하게 인식하여 심각한 의미 오류 발생.

📍 해결책: Positional Encoding

1. 위치 정보 주입

각 단어의 임베딩 벡터에 해당 단어의 '위치'를 나타내는 고유한 벡터를 더해줌으로써 순서 정보를 제공.

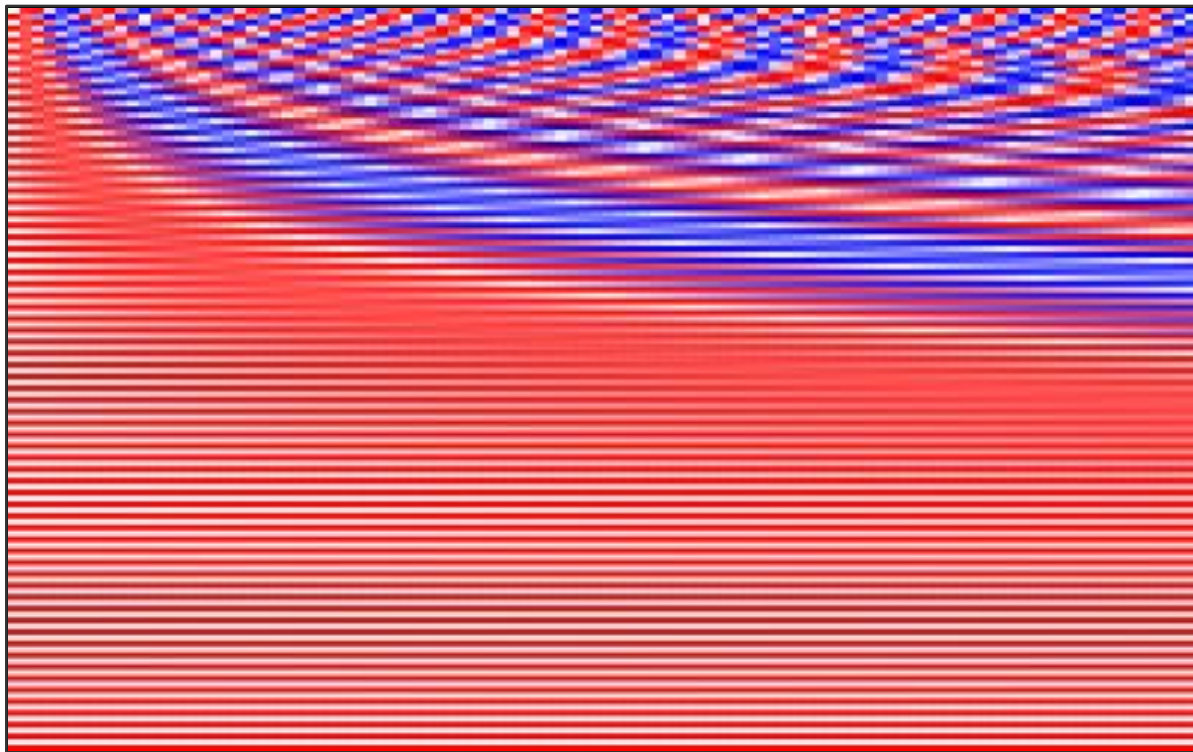
2. 입력 벡터 재정의

입력 = 단어 임베딩 + 위치 임베딩

모델은 이제 단어의 의미뿐만 아니라 문장 내에서의 위치 정보까지 함께 학습하게 됨.

위치 정보 인코딩 (Positional Encoding)

Positional Encoding 패턴 시각화 (d_model=128)



Position Index (Sequence Length)

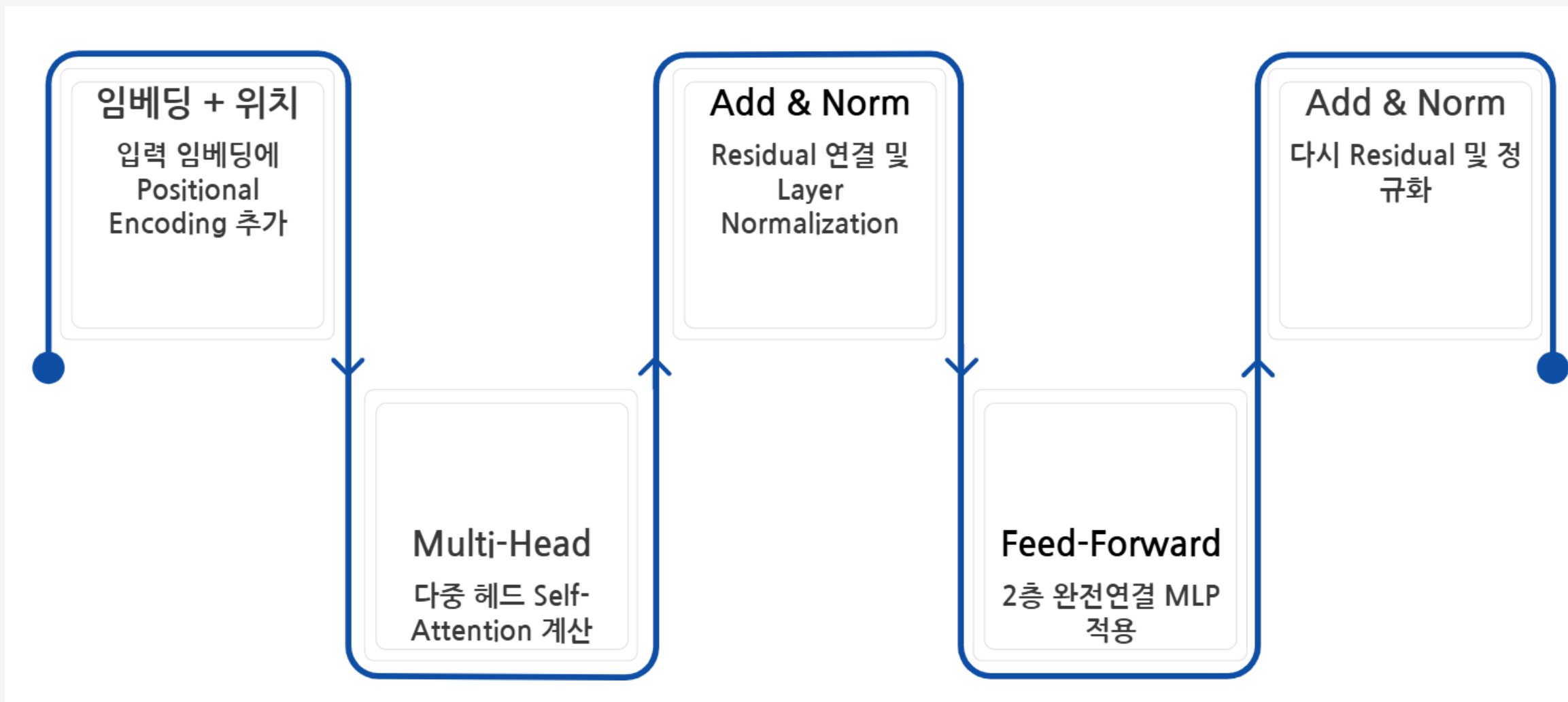
-1.0 (Sin/Cos Min)  +1.0 (Max)

? 왜 Sin/Cos 함수인가?

-  **주기성 (Periodicity)**
비슷한 거리에 있는 단어들은 비슷한 패턴을 가져야 함.
-  **상대 위치 (Relative Position)**
선형 변환만으로 두 위치 간의 거리를 계산할 수 있음.
-  **외삽 (Extrapolation)**
학습 때 보지 못한 긴 문장 길이도 처리 가능.

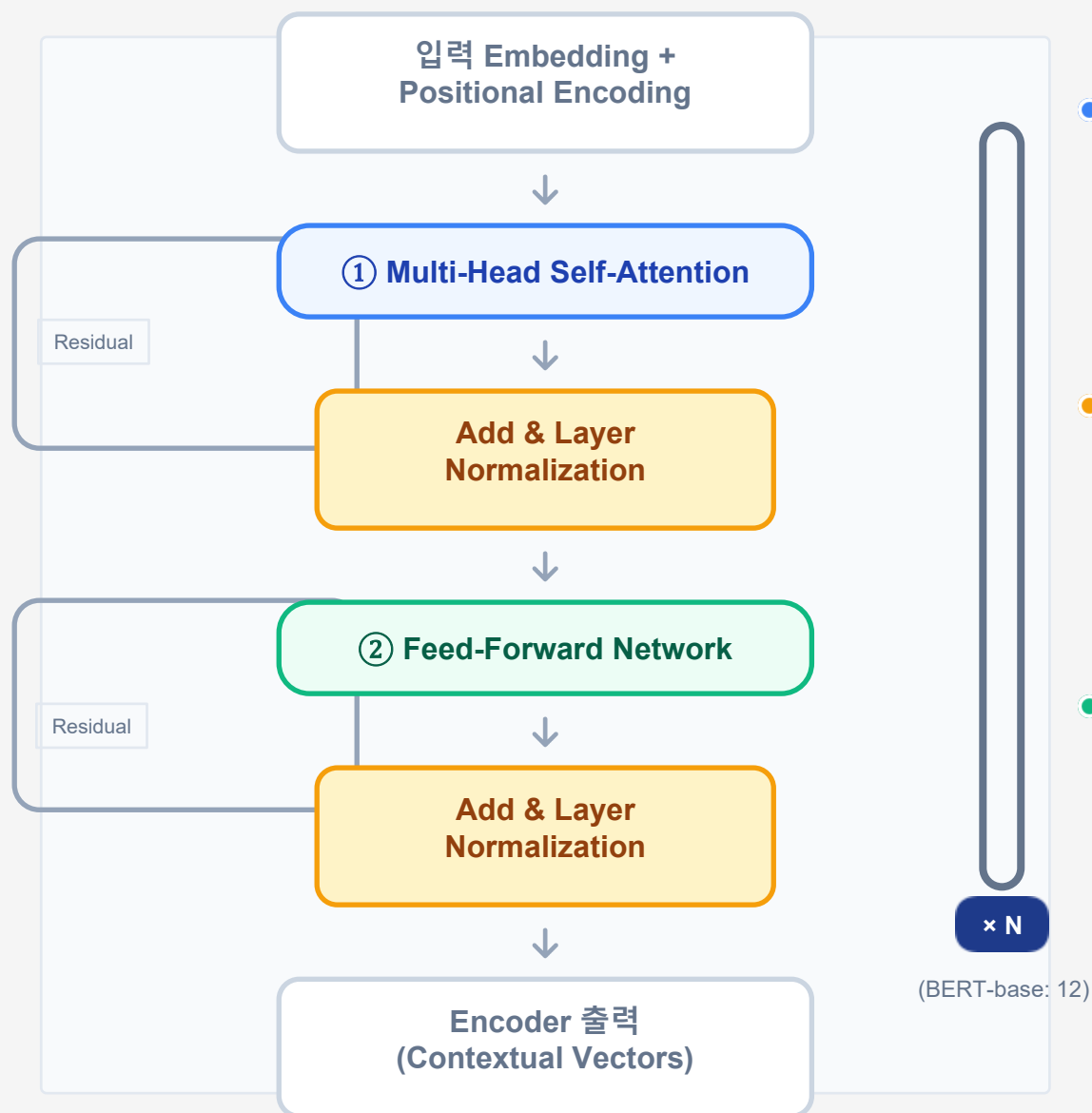
Learned PE vs Fixed PE 비교

구분	Learned PE	Fixed (Sin/Cos) PE
방식	파라미터로 학습	수학적 함수 고정
외삽성	어려움 (길이 제한)	가능 (함수 계산)
성능	비슷함	비슷함
대표 모델	BERT, GPT	Original Transformer



Transformer Encoder 블록 구조

4가지 핵심 구성 요소와 데이터 흐름 완벽 분석



🔍 ① Multi-Head Self-Attention

입력 문장의 모든 단어 간 관계를 학습하여 문맥을 파악합니다.
'it'이 무엇을 가리키는지 등을 알아내는 핵심 단계입니다.

Context Modeling

Global Dependency

+ Add & Layer Normalization

Residual Connection (Add): 하위 층의 정보를 상위 층으로 직접 전달하여 Gradient 소실 문제를 해결합니다.
Layer Norm: 학습을 안정화하고 수렴 속도를 높입니다.

🔧 ② Feed-Forward Network (FFN)

각 위치마다 독립적으로 적용되는 2층 신경망입니다.
비선형성을 추가하고 차원을 확장하여 더 복잡한 패턴을 학습합니다.

Shape: 768 → 3072 → 768 (4배 확장)

📦 Encoder Stack (x N)

이 블록을 N번 반복하여 쌓습니다.
층이 깊어질수록 더 추상적이고 고차원적인 언어적 특징을 학습하게 됩니다.

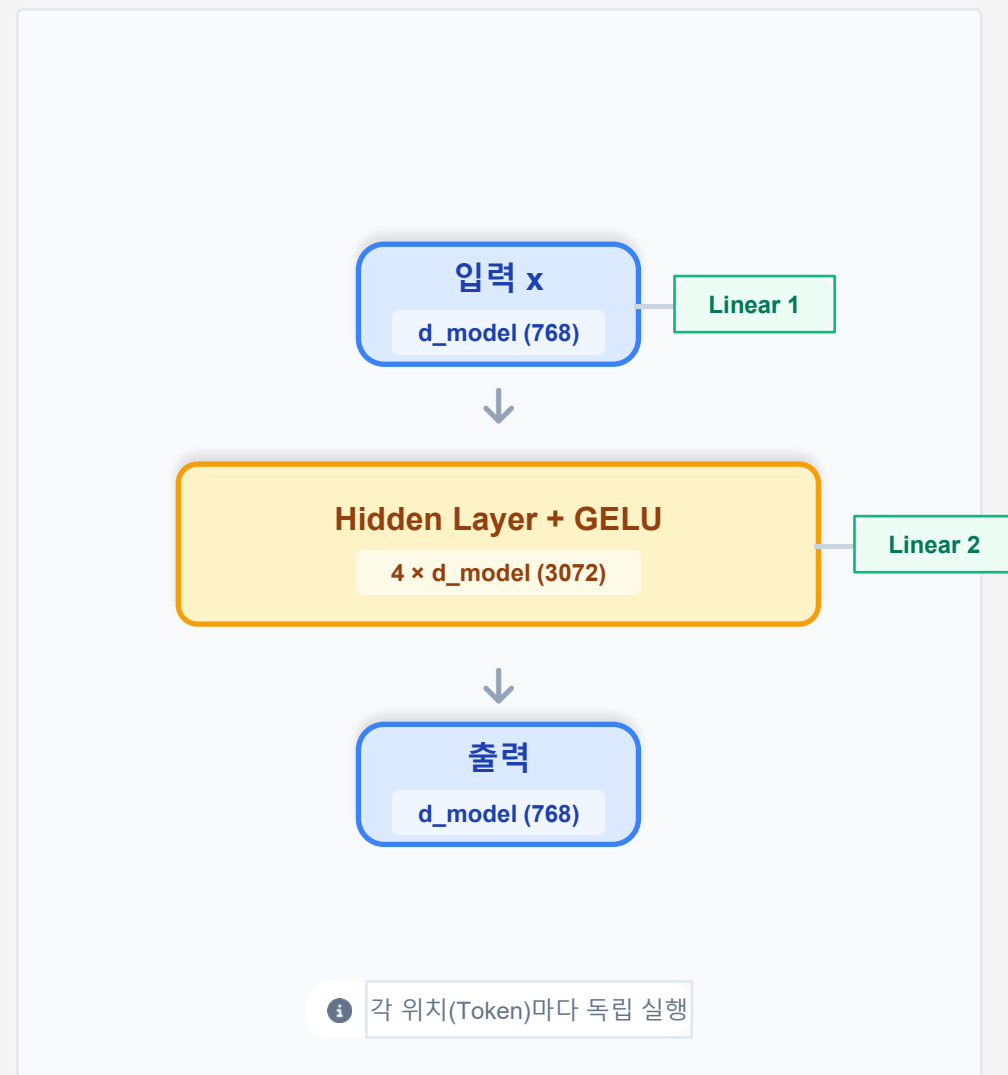
BERT-Base: 12 layers

GPT-3: 96 layers

Transformer Encoder 블록 구조

Feed-Forward Network (FFN) 상세

Attention만으로는 부족하다! 비선형성과 표현력의 핵심



$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

x : 입력 벡터 W_1 : 확장 (768→3072) W_2 : 축소 (3072→768)



왜 4배로 확장하는가? (Expansion)

저차원(768) 공간의 정보를 고차원(3072)으로 투영하여 더 복잡하고 풍부한 특징을 학습합니다. ReLU/GELU 비선형 함수가 이 고차원 공간에서 작동하며 표현력을 극대화합니다.



Position-wise (위치별 독립 처리)

FFN은 문장의 각 단어(위치)마다 개별적으로, 그리고 동일하게 적용됩니다. 즉, 단어 간의 상호작용은 Attention에서 끝나고, FFN은 각 단어의 의미를 심화시키는 역할을 합니다. (병렬화 가능)

파라미터 수 계산 (BERT-base 기준)

W_1 (768 × 3072)	2,359,296
b_1 (3072) + b_2 (768)	3,840
W_2 (3072 × 768)	2,359,296

Total Parameters

≈ 4,722,432 (약 470만)

핵심 구성 요소

1. Multi-Head Self-Attention: 입력 시퀀스 내 모든 위치 간 관계 모델링
2. Residual Connection: 입력을 출력에 더하여 gradient flow 개선
3. Layer Normalization: 각 층의 출력을 정규화하여 학습 안정화
4. Feed-Forward Network: 위치별로 독립적인 2층 MLP 적용
($d_{\text{model}} \rightarrow 4 \times d_{\text{model}} \rightarrow d_{\text{model}}$)

설계 철학

- Residual connection과 Layer Normalization은 **깊은 네트워크 학습을 가능**하게 합니다.
- BERT는 12~24층, GPT-3는 96층의 Encoder/Decoder를 쌓아 고수준 표현을 학습합니다.
- FFN은 attention으로 모은 정보를 **비선형 변환**하여 복잡한 패턴을 포착합니다.
- ReLU나 GELU 활성화 함수를 사용합니다.

미래 정보를 차단하는 Masked Attention과 순차적 생성 과정

Encoder vs Decoder 차이점		
구분	Encoder	Decoder
Self-Attention	양방향 (전체 참조)	Masked (과거만 참조)
Cross-Attention	없음 (X)	있음 (Encoder 참조)
역할	입력 이해 (understanding)	출력 생성 (Generation)

Cross-Attention (Encoder-Decoder)

Decoder가 생성을 할 때
Encoder의 정보를 참고하는 단계입니다.

Query(Q): Decoder의 현재 상태
Key(K), Value(V): Encoder의 최종 출력

Masked Attention 시각화

"I love you" 문장 생성 시 참조 가능 영역

	I	love	you	<E>
I	1	-∞	-∞	-∞
love	1	1	-∞	-∞
you	1	1	1	-∞
<E>	1	1	1	1

● 참조 가능 (Visible) ● 미래 정보 차단 (Masked)

Why Masking?

학습 시 정답(미래 단어)을 미리 보고 예측하는 '커닝(Cheating)' 방지

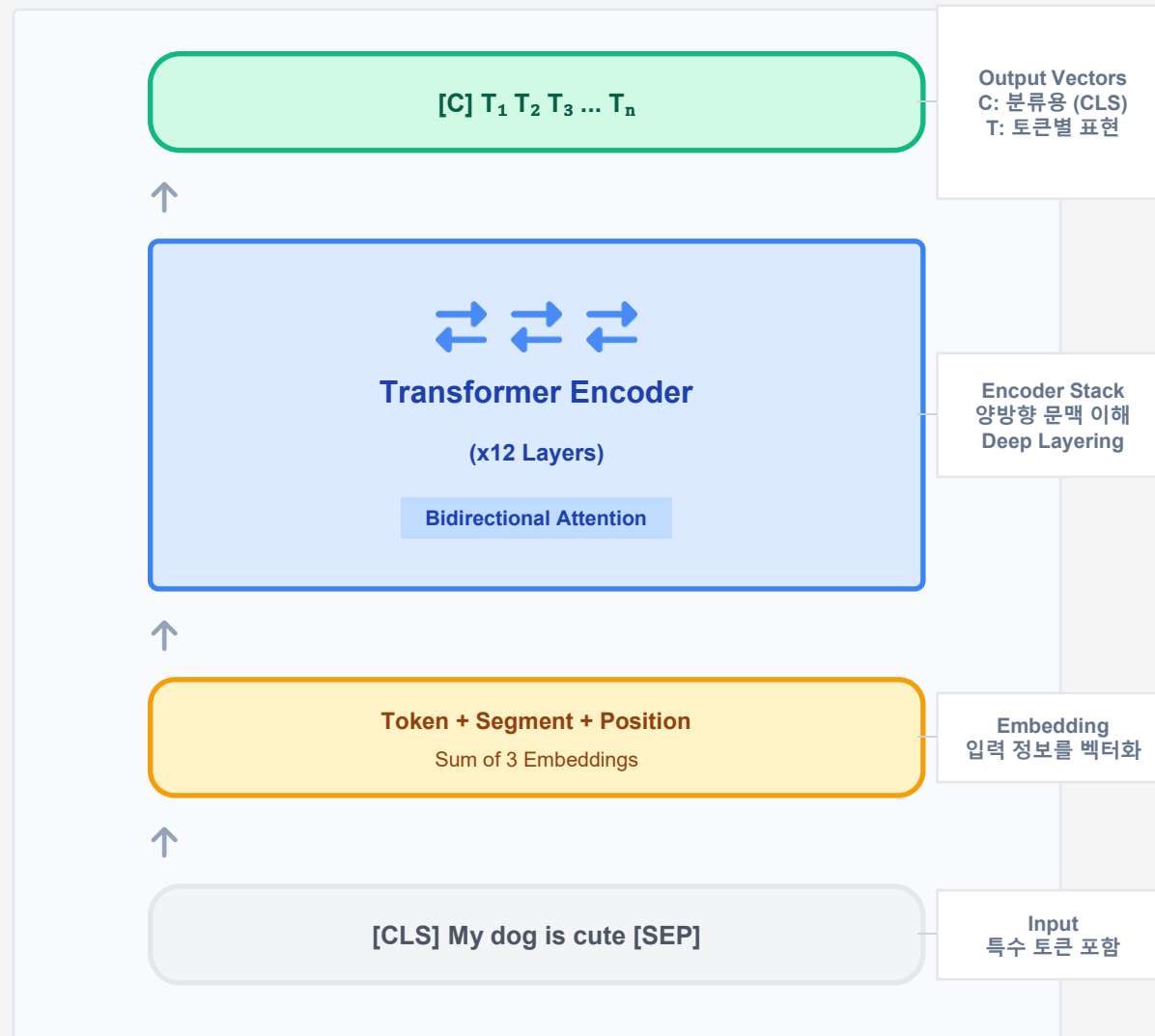
Auto-regressive 생성



이전 단계의 출력이 다음 단계의 입력으로 다시 들어가는 구조

BERT와 파인튜닝: Transfer Learning의 핵심

BERT 구조와 사전학습 Bidirectional Encoder Representations from Transformers



Encoder-only & Bidirectional

BERT는 Transformer의 **Encoder** 부분만 사용하여, 문장의 앞뒤 문맥을 동시에 파악하는 **양방향(Bidirectional)** 모델입니다. (GPT는 단방향 Decoder)

Task 1: MLM

Input: I [MASK] you.
Label: love

Masked Language Model

문장의 빈칸(15%)을 뚫고, 문맥을 통해 해당 단어를 맞추는 과정에서 언어 이해력 학습.

Task 2: NSP

Sent A: I love you. Sent B: He flies. (IsNext?)

Next Sentence Prediction

두 문장이 이어지는 문장인지 판단하며 문장 간의 관계 학습.

모델 (Model)	Layers	Hidden Size	Heads	Parameters
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M

* 파라미터 수: 학습된 가중치의 총 개수



사전학습 (Pre-training)

대규모 텍스트 코퍼스(Wikipedia, BookCorpus)에서 Masked Language Model(MLM)과 Next Sentence Prediction(NSP) 태스크로 학습합니다. MLM은 문장의 15% 토큰을 마스킹하고 원래 단어를 예측하도록 훈련합니다.



파인튜닝 (Fine-tuning)

사전학습된 BERT에 태스크별 분류 헤드를 추가하고 소량의 라벨 데이터로 fine-tuning합니다. 감정 분석, 개체명 인식, 질의응답 등 다양한 downstream task에 적용 가능합니다.



DistilBERT의 효율성

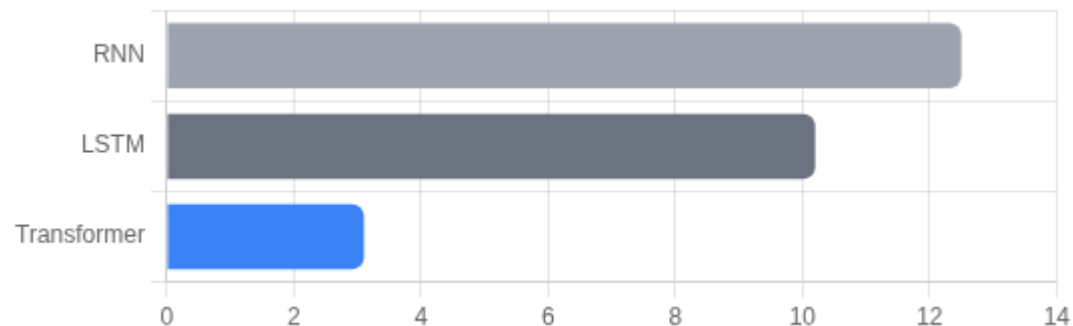
BERT의 지식을 증류(distillation)하여 파라미터를 40% 줄이고 속도를 60% 향상시킨 경량화 모델입니다. 성능은 BERT의 97% 수준을 유지하며 실시간 서비스에 적합합니다.

Transformer vs RNN/LSTM 비교

순차적 처리의 한계를 넘어 병렬 연산의 시대로

특징	RNN / LSTM	Transformer
병렬처리	불가능(순차계산)	완벽함(동시 계산)
장기의존성	어려움(Gradient 소실)	쉬움
학습속도	느림	매우 빠름(GPU에 최적화)
계산복잡도	$O(n)$ (선형적, 가벼움)	$O(n^2)$ (길이에 제곱 비례)
해석가능성	어려움(black box)	쉬움(Attention map)
문맥길이	짧음(정보손실 발생)	김(모든 토큰 참조)

학습 소요 시간 (Training Time) (낮을수록 좋음)



병렬 처리 덕분에 Transformer가 압도적으로 빠름

번역 성능 (BLEU Score) (높을수록 좋음)

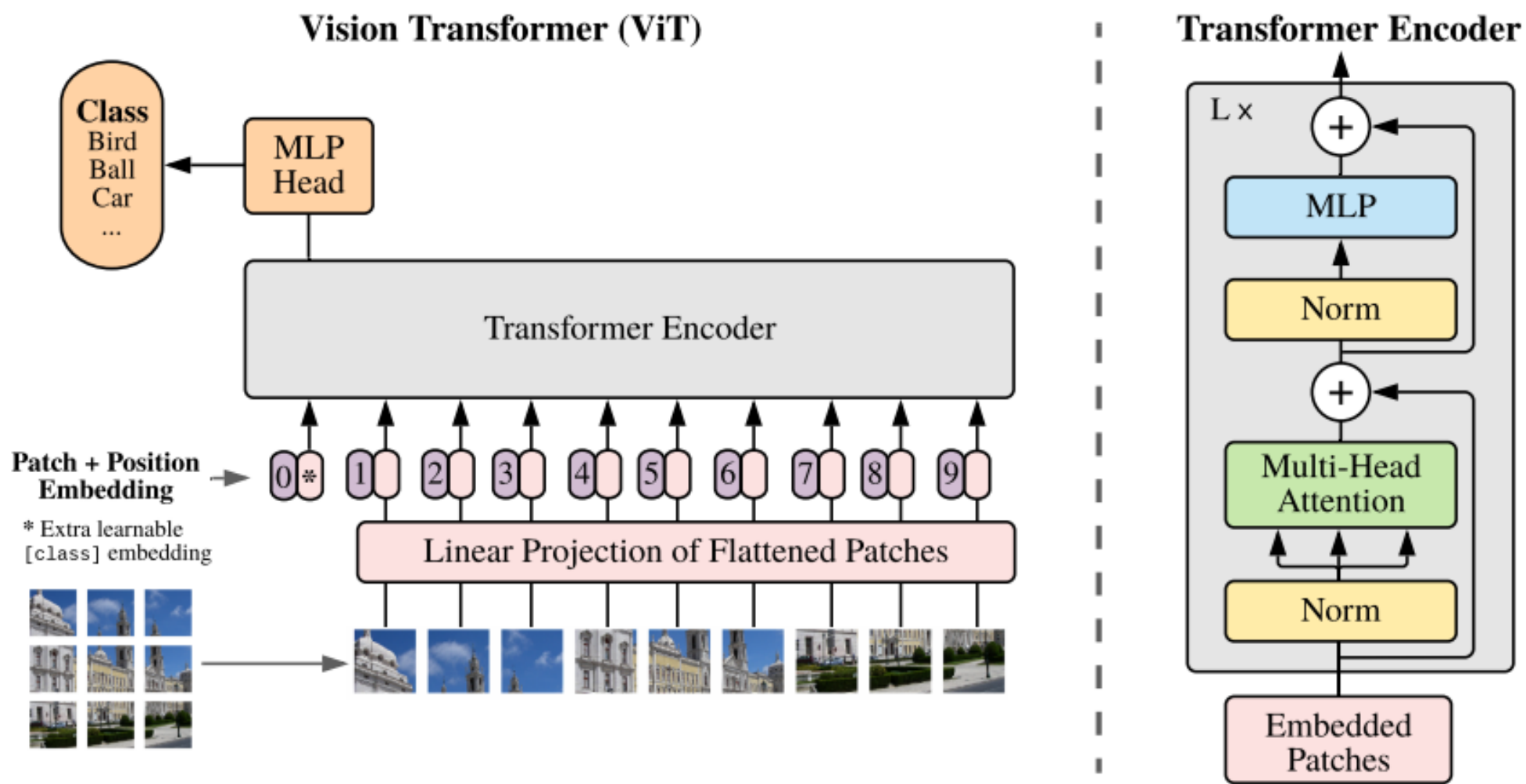


장거리 문맥 파악으로 번역 품질 향상

Vision Transformer (ViT): 이미지를 시퀀스로

ViT는 **CNN의 inductive bias(locality, translation equivariance)** 없이
순수 attention으로 이미지를 처리합니다.

대규모 데이터(ImageNet-21K 이상)에서 사전학습하면 CNN을 능가하는 성능을 보입니다.



Vision Transformer (ViT): 이미지를 시퀀스로

이미지를 패치 시퀀스로 처리하는 혁신적 접근



이미지 = 패치 시퀀스

이미지를 16x16 크기의 패치로 잘라 단어(Token)처럼 처리합니다.
CNN 없이 순수 Attention만으로 이미지를 이해합니다.



Inductive Bias 제거

CNN의 Locality(지역성)와 Translation Equivariance(이동 불변성) 가정을 제거
→ 데이터 간의 관계를 스스로 학습합니다.



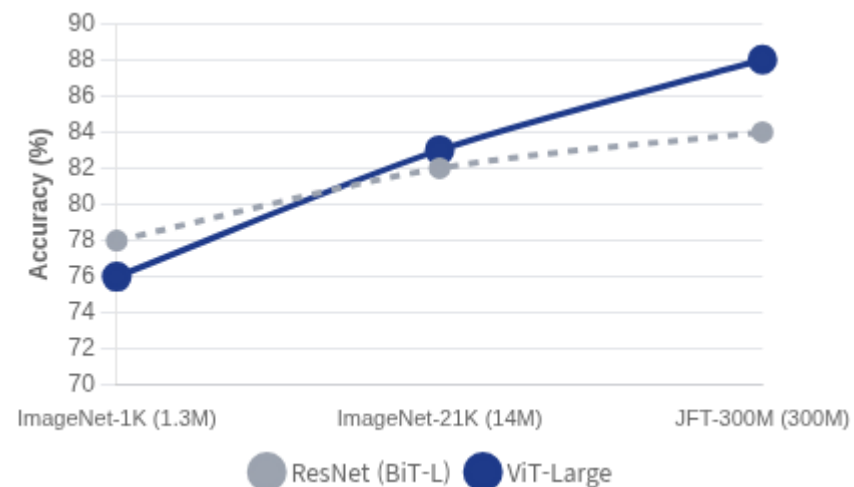
데이터 규모의 승리

소규모 데이터에서는 CNN보다 성능이 낮지만,
대규모 데이터(JFT-300M 등)로 사전학습 시 CNN을 능가합니다.

"An Image is Worth 16x16 Words"

(이미지는 16x16 크기의 단어들과 같다 - ViT 논문 제목)

데이터 크기에 따른 성능 비교 (ImageNet Top-1)



* 데이터셋 크기가 커질수록 ViT 성능이 급격히 향상됨

CNN vs ViT 핵심 차이

CNN:

이미지 특화 구조 (Inductive Bias 강함), 적은 데이터로도 학습 가능

ViT:

일반적인 구조 (Inductive Bias 약함), 많은 데이터 필요,
글로벌 문맥 파악 우수

이미지 패치 분할

224×224 이미지를 16×16 크기의 패치로 나누면 14×14=196개의 패치가 생성됩니다. 각 패치는 16×16×3=768차원 벡터로 flatten됩니다.

선형 투영 및 위치 임베딩

각 패치를 선형 변환하여 d_{model} 차원(768)으로 매핑하고, 학습 가능한 positional embedding을 더합니다. 추가로 CLS 토큰을 앞에 붙여 분류에 사용합니다.

Transformer Encoder 적용

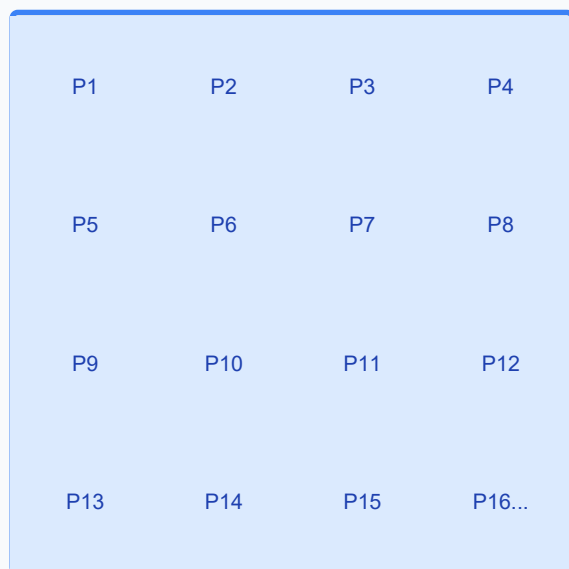
BERT와 동일한 Transformer Encoder를 통과시킵니다. ViT-Base는 12층, 12 heads, 768 hidden dim을 사용하며, 최종 CLS 토큰의 표현을 분류 헤드에 입력합니다.

Vision Transformer (ViT): 이미지를 시퀀스로

ViT 입력 처리 5단계

이미지를 Transformer가 이해할 수 있는 "문장"으로 변환하는 과정

Step 1: 16×16 Patches



Original Image

$224 \times 224 \times 3$ (RGB)

Patch Size

16×16

Grid Size

$14 \times 14 = 196$ patches

Vector Dim

$16 \times 16 \times 3 = 768$

1

이미지 패치 분할

이미지를 고정된 크기(16x16)의 패치들로 자릅니다.

$N = 14 \times 14 = 196$

2

Flatten (평탄화)

각 패치를 1차원 벡터로 펼칩니다. ($16 \times 16 \times 3 = 768$)

[196, 768]

3

Linear Projection

학습 가능한 선형 층을 통과시켜 임베딩 벡터로 변환합니다.

Dim: 768 $\rightarrow$ 768

4

Position Embedding 추가

패치의 원래 위치 정보를 더해줍니다. (순서 정보 제공)

+ Pos[196, 768]

5

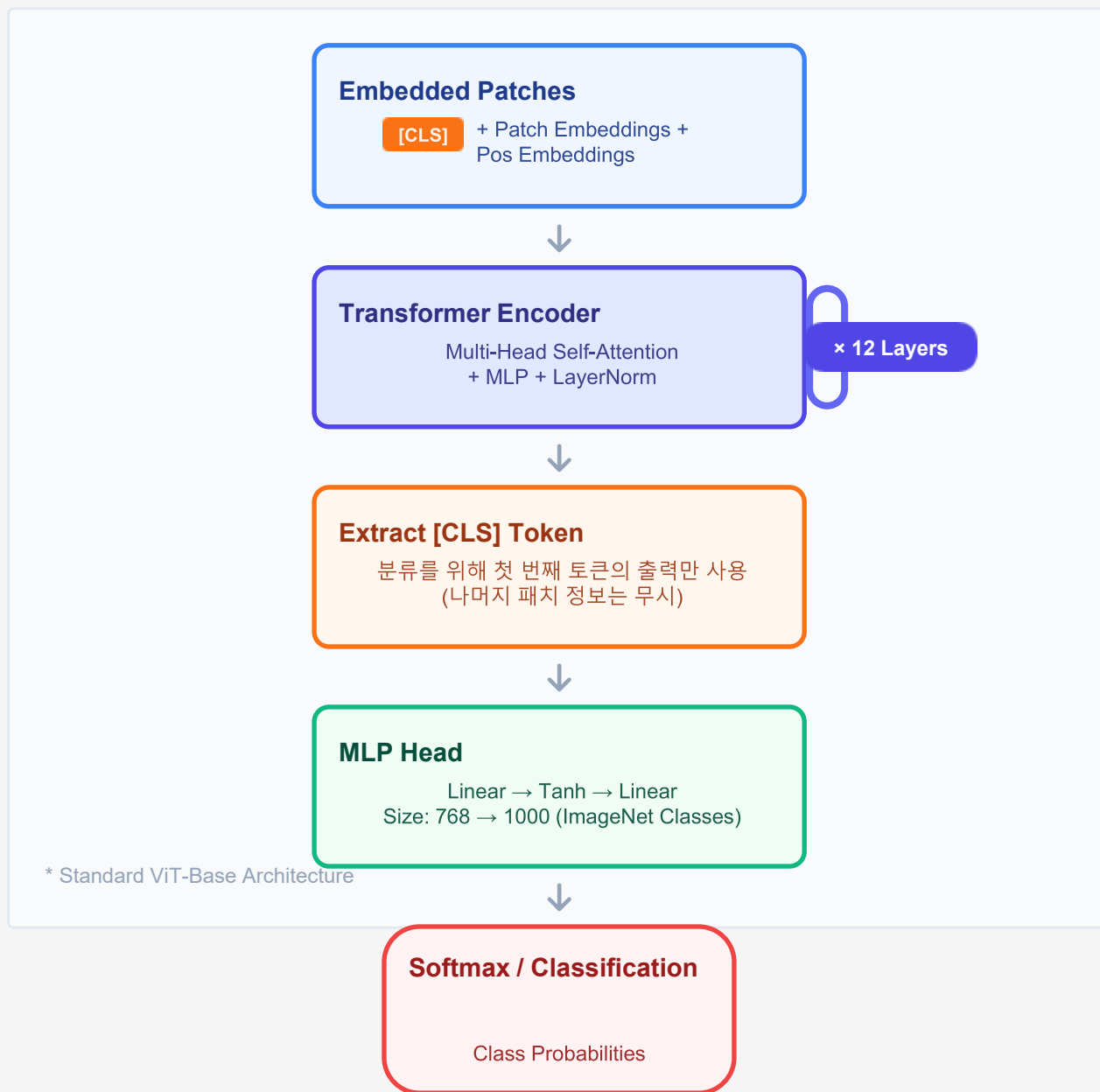
[CLS] Token 추가

맨 앞에 학습 가능한 분류 토큰을 추가합니다.

Total: 197 tokens

Vision Transformer (ViT) 아키텍처

Transformer Encoder 기반의 이미지 분류 모델 구조



ViT-Base Specification

Model Configuration

	16 × 16
	768
	12
	12
	3,072
	86 Million

i BERT-Base 모델과 거의 동일한 하이퍼파라미터를 사용하여, NLP에서의 성공 방정식을 Vision에 그대로 적용했습니다.

Vision Transformer (ViT): ViT Attention Map

층(Layer)이 깊어질수록 지역 정보에서 전역 문맥으로 시야가 확장됩니다



[CLS] Token의 역할

최종 분류를 담당하는 [CLS] Token은 마지막 레이어에서 이미지의 가장 특징적인 부분(Discriminative Region)에 강하게 Attention을 집중합니다.

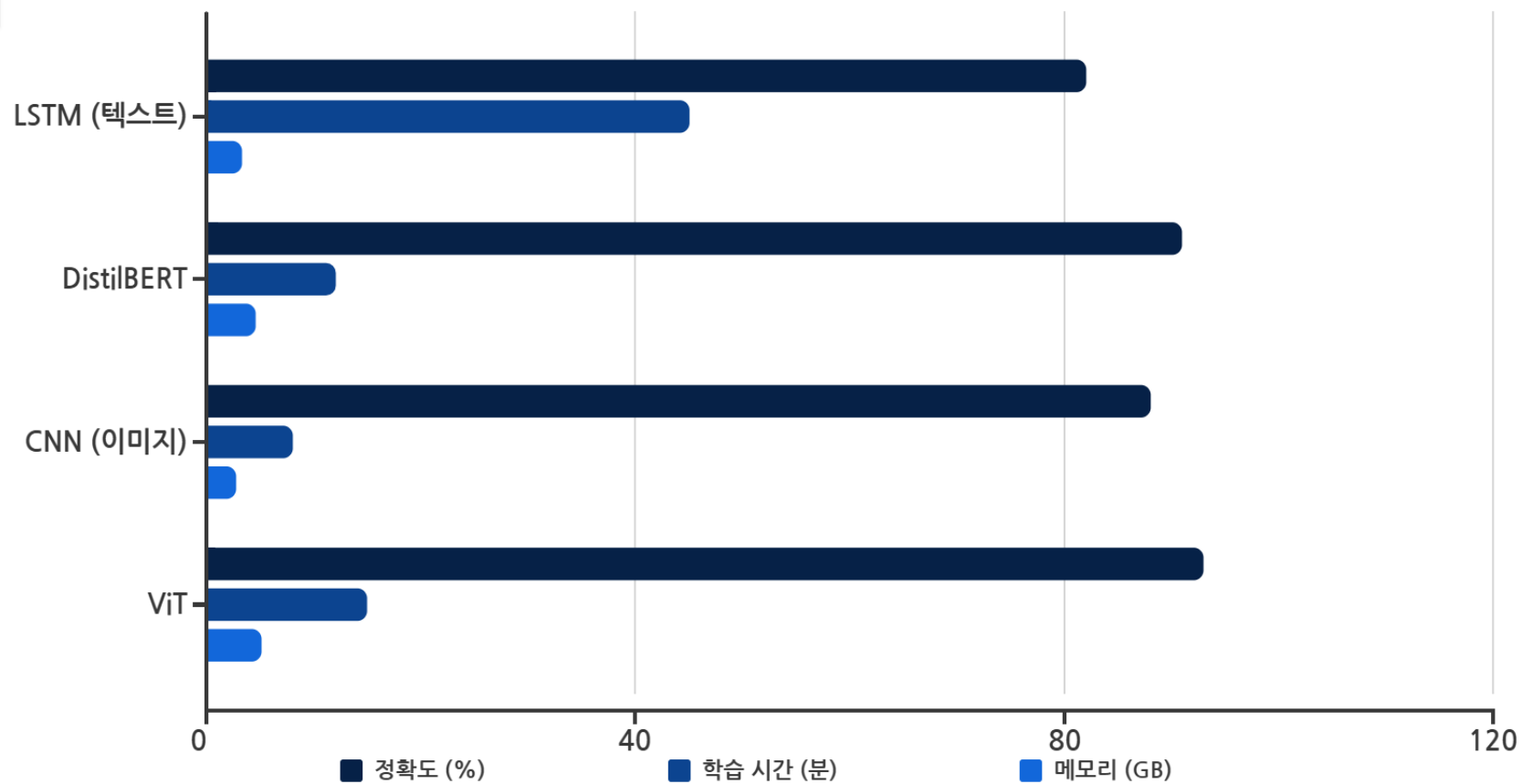
💡 CNN은 고정된 필터로 주변만 보지만, ViT는 필요한 곳을 스스로 선택해서 봅니다.

Attention Distance vs Network Depth



* 층이 깊어질수록 평균 Attention 거리(참조 범위)가 증가함

모델 성능비교 : LSTM vs Transformer / CNN vs ViT



정확도

Transformer 계열이 9~11%p 높은 정확도를 기록했습니다. Self-Attention의 전역적 문맥 파악 능력이 핵심 요인입니다.

학습 시간

DistilBERT는 LSTM 대비 3.7배 빠르며, 병렬 처리 최적화 덕분입니다. ViT는 CNN보다 약간 느리지만 대규모 데이터에서는 역전됩니다.

메모리 사용량

Transformer는 attention matrix($O(n^2)$)로 인해 메모리가 많이 필요합니다. 긴 시퀀스 처리 시 gradient checkpointing 등 최적화 필수입니다.

CNN vs ViT 언제 무엇을 쓸까?

데이터 규모와 목적에 따른 모델 선택 전략 및 성능 비교

핵심 특징 비교

Feature	CNN (ResNet)	ViT (Transformer)
Inductive Bias	강함 Locality & Invariance	약함 데이터에서 관계 학습
데이터 요구량	적음 ImageNet-1K 충분	매우 많음 ImageNet-21K / JFT 권장
학습 효율성	빠름 하드웨어 최적화 우수	느림 많은 연산량 ($O(N^2)$)
해석 가능성	어려움 Feature Map 추상적	쉬움 Attention Map 시각화
전이 학습	준수함	매우 우수 대규모 사전학습 시 강력

CNN을 선택하세요

- ✓ 데이터가 적을 때 (< 1M images)
- ✓ 빠른 학습/추론이 필요할 때
- ✓ 모바일/엣지 디바이스 배포
- ✓ 사전학습 없이 처음부터 학습할 때

ViT를 선택하세요

- ✓ 초대규모 데이터셋 확보 가능 시
- ✓ 최고 수준(SOTA)의 성능이 필요할 때
- ✓ 대규모 사전학습 모델 파인튜닝 시
- ✓ 모델의 판단 근거(Attention)가 중요할 때

	ViT 계열 (Vision)				
Model	Patch Size	Layers	Hidden	Heads	Params
ViT-Base	16×16	12	768	12	86M
ViT-Large	16×16	24	1024	16	307M
ViT-Huge	14×14	32	1280	16	632M

Transformer 핵심 4대장



Self-Attention

모든 위치 간 직접 연결 및 문맥 파악



Positional Encoding

순서 정보가 없는 집합 연산에 위치 부여



Multi-Head

다양한 관점(문법, 의미, 위치) 동시 학습



Parallelization

순차 처리 탈피, GPU 병렬 연산 극대화



응용 분야 확장 (Evolution)

Transformer Architecture

NLP

BERT 이해 (Encoder)

GPT 생성 (Decoder)

T5 통합 (Enc-Dec)

Vision

ViT 이미지 분류

DETR 객체 검출

Swin 계층적 구조

Multi-modal

CLIP 이미지+텍스트

Flamingo Vision-Lang